



UNIVERSITÀ  
DEGLI STUDI  
DI TRIESTE

# Generalized Reserve Set Covering Problem with Buffer and Connectivity Requirements

---

Camilla Bigotto, Anna Guccione

# Introduction

---

# The GRSC-CB problem

---

## DEFINITION OF THE PROBLEM

- Design of nature reserves for ensuring the conservation of endangered wildlife
- Reserves must verify a series of spatial requirements:
  - **Connectivity:** to avoid spatial fragmentation
  - **Buffer zones:** surrounding/protecting the core areas
- **General Reserve Set Covering Problem with Connectivity and Buffer Requirements**

*Original authors of the paper: Álvarez-Miranda, Goycoolea, Ljubić, Sinnl (European Journal of Operational Research, 2021), available at <https://www.sciencedirect.com/science/article/abs/pii/S0377221719305818>*

# The GRSC-CB problem

---

## MODULAR APPROACH

| Model          | Description                                            |
|----------------|--------------------------------------------------------|
| <b>RSC</b>     | Minimum sites covering all species                     |
| <b>GRSC</b>    | + suitability quota, species groups, cost minimization |
| <b>GRSC-B</b>  | + buffer zone constraints                              |
| <b>GRSC-C</b>  | + connectivity constraints                             |
| <b>GRSC-CB</b> | B + C (full model)                                     |

- Branch and cut framework
- Heuristics:
  - Construction
  - Primal
  - Local-branching

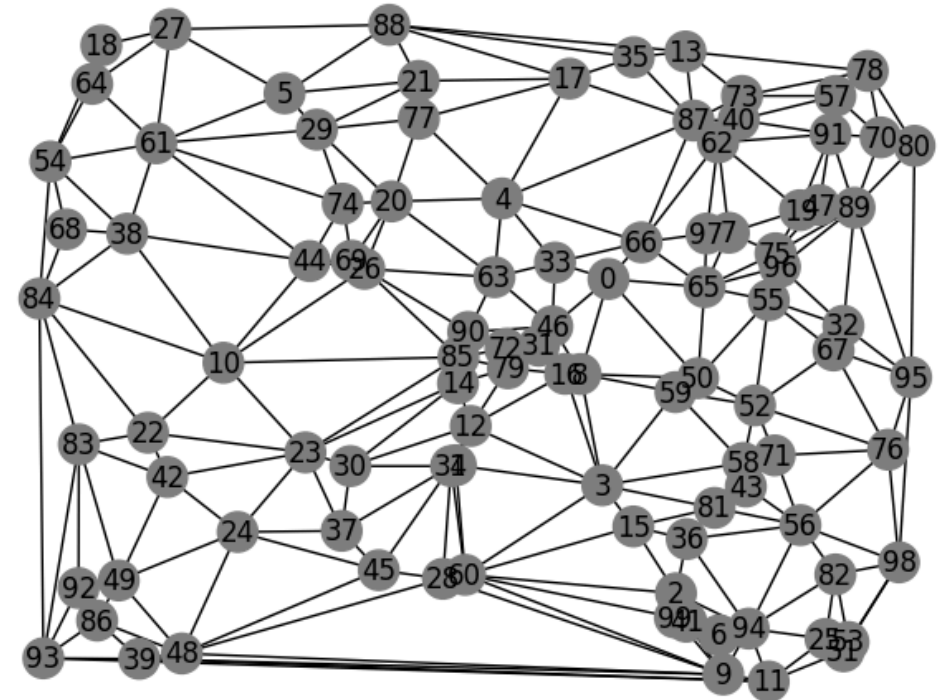
# Generalized Reserve Set Covering Problem

---

# RSC: Reserve Set Covering problem

## DEFINITIONS

- Given:
  - A set  $V$  of **land sites**
  - A set of **species**  $S$   
 $\forall s \in S$  a set of suitable land sites  $V_s \subseteq V$
- Find the minimum number of reserve sites such that each species is present in the selected set of sites at least once
- A **graph** was considered with:
  - as **nodes** the land sites
  - an **edge** is present between 2 land sites if they share a border



# RSC: Reserve Set Covering problem

---

## Variable

$x_i$  with  $1 \leq i \leq |V|$  defined as:

$$x_i = \begin{cases} 1, & \text{if the land site } i \in V \text{ is selected} \\ 0, & \text{otherwise} \end{cases}$$

## Constraint

### COV

Each species needs to be covered by a suitable land site

$$\sum_{i \in V_s} x_i \geq 1, \forall s \in S$$

## Objective Function

**Minimize** the number of land sites

$$\min \pi(x) = \sum_{i \in V} x_i$$

# GRSC: Generalized RSC

---

## DEFINITIONS

### Cost

- $c : V \rightarrow \mathbb{R}^+ \rightarrow$  **cost function** of choosing the site  $i \in V$  to be in the reserve

### Quotas of ecological suitability

- $w : V \times S \rightarrow \mathbb{R}^+ \rightarrow$  **habitat suitability function** measures suitability of  $i \in V$  for a species  $s \in S$
- $\lambda_s \geq 0 \rightarrow$  **minimum quota of ecological suitability** for  $s$
- $s \in S$  is considered **protected** if  $\sum_{i \in V_s} w(i, s) \geq \lambda_s$

### Species

- $S_1 \subseteq S \rightarrow$  set of species that need to stay in the core area
- $S_2 = S \setminus S_1 \rightarrow$  can live anywhere
- $P_1$  and  $P_2$ : minimum number of species respectively in  $S_1$  and  $S_2$  that the reserve needs to protect



# GRSC: Generalized RSC

---

## Variables

- $x_i$  defined as before
- $z_i, 1 \leq i \leq |V| \rightarrow$  binary variables defined as:
$$z_i = \begin{cases} 1, & \text{if the land site } i \in V \text{ is part of the core area} \\ 0, & \text{otherwise} \end{cases}$$
- $u_s, 1 \leq s \leq |S| \rightarrow$  binary variables defined as:
$$u_s = \begin{cases} 1, & \text{if the species } s \in S \text{ is protected by the reserve} \\ 0, & \text{otherwise} \end{cases}$$

The solution will be a tuple  $(u, x, z)$

# GRSC: Generalized RSC

---

## Constraints

### S1-SQ

If a species in  $S_1$  is protected, the habitat suitability score for the land sites part of the core area must be at least the minimum quota of ecological suitability of that species

$$\sum_{i \in V_s} w(i, s) z_i \geq \lambda_s u_s \quad \forall s \in S_1$$

### S2-SQ

Similarly for a species in  $S_2$ , considering land sites in the whole reserve

$$\sum_{i \in V_s} w(i, s) x_i \geq \lambda_s u_s \quad \forall s \in S_2$$

# GRSC: Generalized RSC

---

## Constraints

### S1-PROTECT

At least  $P_1$  species of  $S_1$  must be protected

$$\sum_{s \in S_1} u_s \geq P_1$$

### S2-PROTECT

At least  $P_2$  species of  $S_2$  must be protected

$$\sum_{s \in S_2} u_s \geq P_2$$

# GRSC: Generalized RSC

---

## Constraints

### LINK

If a site is in the core, then it must also be in the reserve

$$z_i \leq x_i \quad \forall i \in V$$

## Objective function

### COST

Function that indicates the cost that needs to be **minimized**

$$\gamma(u, x, z) = \sum_{i \in V} c_i x_i$$

# GRSC: Generalized RSC

---

## GRSC MODEL

$$GRSC : \min\{\gamma(u, x, z) | \\ (S1-SQ), (S2-SQ), (S1-PROTECT), (S2-PROTECT), (LINK)\}$$

# Buffer and Connectivity requirements

---

# GRSC-B: GRSC with Buffer requirements

---

## $d$ -neighbourhood

- To model the buffer surrounding the selected core areas, the  $d$ -neighbourhood set of a node  $i$  is defined
- For  $d \geq 0, d \in \mathbb{N}, i \in V$  the  $d$ -neighbourhood is defined as the set of sites at distance  $d$  from  $i$   
$$\delta_d(i) = \{j \in V_{i \neq j} \mid \text{the number of jumps from } i \text{ and } j \text{ is } \max d\}$$
- $\delta_d^+(i)$  is the  $d$ -neighbourhood that contains also  $i$

*In the implementation: the distances are precomputed using `nx.ego_graph` and stored as a `frozenset`*

# GRSC-B: GRSC with Buffer requirements

---

## Constraints

### d-BUFF.1

Every core site is surrounded by a buffer area of normal sites in its  $d$ -neighbourhood

$$z_i \leq x_j, \quad \forall j \in \delta_d(i), \quad \forall i \in V$$

### d-BUFF.2

Every normal site must have at least one core site in the  $d$ -neighbourhood

$$x_i \leq \sum_{j \in \delta_d(i)} z_j \quad \forall i \in V$$



# GRSC-B: GRSC with Buffer requirements

---

## GRSC-B MODEL:

$$\begin{aligned} GRSC-B : \min \{ & \gamma(u, x, z) | \\ & (S1-SQ), (S2-SQ), (S1-PROTECT), (S2-PROTECT), (LINK) \\ & (d-BUFF.1), (d-BUFF.2) \} \end{aligned}$$

# GRSC-C: GRSC with Connectivity requirements

---

## $r$ -arc-node-separator

- To model connectivity, a root  $r$  is added to the graph, obtaining  $G_r$ , where each node is connected to the root with an arc
- The arcs  $(r, i)$  are called  $r$ -**arcs** ( $A_r$ )
- Given a node  $\ell$ , an  $r$ -**arc-node-separator** ( $W = (W_V, W_A)$ ) is the set of nodes and arcs such that if they're removed from  $G_r$ , then the site  $\ell$  can't be reached by  $r$
- $W_\ell$  is the set of all  $r$ -arc-node-separators w.r.t  $\ell$
- **Connectivity requirement:** every site must have a path from  $r$

# GRSC-C: GRSC with Connectivity requirements

---

## Variables

- $(u, x, z)$  defined as before
- Let  $y_i, 1 \leq i \leq |V|$  be an auxiliary variable
$$y_i = \begin{cases} 1, & \text{if the land site } i \in V \text{ is connected to } r \text{ via an arc } (r, i) \\ 0, & \text{otherwise} \end{cases}$$
- Each  $y_i$  represents a **connected component root**

## Constraints

Let  $k$  be the max number of connected components in the reserve

### NCOMP

There's a max of  $k$  connected components in the reserve

$$\sum_{j \in V} y_j \leq k$$

# GRSC-C: GRSC with Connectivity requirements

---

## Constraints

### YX

If a site  $i$  is connected to the root, then it must be considered in the reserve area

$$y_i \leq x_i \quad \forall i \in V$$

### ALLCON

Every selected parcel in the reserve must be connected to the artificial root through at least one element of its corresponding r-arc-node-separators (one node from  $W_V$  or one arc from  $W_A$ )

$$\sum_{i \in W_V} x_i + \sum_{j \in W_A} y_j \geq x_\ell, \quad \forall W \in W_\ell, \quad \forall \ell \in V$$

# GRSC-C: GRSC with Connectivity requirements

---

## GRSC-C MODEL:

$$\begin{aligned} GRSC-C : \min \{ & \gamma(u, x, z) | \\ & (S1-SQ), (S2-SQ), (S1-PROTECT), (S2-PROTECT), (LINK) \\ & (ALLCON), (YX), (NCOMP) \} \end{aligned}$$

# GRSC-CB: GRSC with Buffer and Connectivity

---

## Constraints

### YZ (replaces YX)

If a site  $i$  is connected to the root, then it must be considered in the **core** area

$$y_i \leq z_i \quad \forall i \in V$$

### CORECON (replaces ALLCON)

Every selected parcel in the **core** of the reserve must be connected to the artificial root through at least one element of its corresponding r-arc-node-separators (one node from  $W_V$  or one arc from  $W_A$ )

$$\sum_{i \in W_V} z_i + \sum_{j \in W_A} y_j \geq z_\ell, \quad \forall W \in W_\ell, \quad \forall \ell \in V$$

# GRSC-CB: GRSC with Buffer and Connectivity

---

## GRSC-CB MODEL:

$$\begin{aligned} GRSC-CB : \min \{ & \gamma(u, x, z) | \\ & (S1-SQ), (S2-SQ), (S1-PROTECT), (S2-PROTECT), (LINK) \\ & (d-BUFF.1), (d-BUFF.2) \\ & (CORECON), (YZ), (NCOMP) \\ & (u, x, z, y) \in \{0, 1\}^{|S|+3|V|} \} \end{aligned}$$

# Branch-and-cut framework

---



# Motivations

---

- In the GRSC-CB and GRSC-C, the family of constraints caused by **connectivity cuts** (CORECON and ALLCON) are exponential in number
- Since they must account for every possible  $r$ -arc-node-separator, which grows quickly with the dimension of the instance
- Therefore, a **branch-and-cut** framework is built to separate the connectivity constraints
- **lazy constraints** are added to the model once a violation is found during optimization

*Only CORECON is described, since ALLCON is symmetrical*

# Valid inequalities

- To improve lower bounds and narrow the search space, 3 families of valid inequalities are also separated during the LP relaxation

| Cut                               | Ensure that                                                                          |
|-----------------------------------|--------------------------------------------------------------------------------------|
| <b>COVER</b> (Cover Inequalities) | If a specie is protected, at least one parcel from the $C_s$ is taken in the reserve |
| <b>SC</b> (Species Cuts)          | If a species is protected, at least one node in $V_s$ is connected to the root $r$   |
| <b>SCC</b> (Species - Cover Cuts) | Like <b>SC</b> , but considering $C_s$ instead of $V_s$                              |

- Where the set  $C_s \subset V_s$  (**Cover**) contains the indispensable nodes, such that  $V_s \setminus C_s$  is not enough to satisfy the sustainability quota for that species

*In the implementation, SC cuts were not considered, since SCC are stronger*

# Implementation

---

At each **integer node** ( MIPSOL callback):

1. Separate **CORECON** (integer) → if violated, add cut

At each **LP relaxation node** ( MIPNODE callback):

1. Separate **COVER** and separate **SCC** → if violated, add cut
2. If no violated inequality is found, separate **CORECON** (fractional) → if violated, add cut

Code structure:

```
model.Params.LazyConstraints = 1
...
def callback(where):
    if where == MIPSOL: # → integer solution
        cuts ← separate_CORECON()
        model.cbLazy(cuts as constraint)

    if where == MIPNODE: # → LP relaxation
        cuts ← separate_COVER() + separate_SCC()
        if cuts is empty:
            cuts ← separate_CORECON()
            model.cbLazy(cuts as constraint)
    ...
model.optimize(callback)
```

# CORECON separation

## Integer solution

- $H$  = connected component of the core induced by  $\tilde{z}_i = 1$  that does **not** contain any root arc (i.e.,  $\tilde{y}_i = 0$  for all  $i \in H$ )  $\rightarrow$  the component is disconnected from  $r$

- The connectivity cut is defined as  $(W_V, W_A)$  such that:

$$W_V = \{j \mid \{i, j\} \in E, i \in H, j \notin H\}, W_A = H$$

- For each  $H$ , the lazy constraint added:

$$\sum_{i \in W_V} z_i + \sum_{j \in W_A} y_j \geq z_\ell, \quad \forall W \in W_\ell, \quad \forall \ell \in V$$

*In the implementation: the connected component are computed using the **NetworkX** method `connected_components`*

# CORECON separation

## Fractional solution

- Let  $\rho = (\tilde{u}, \tilde{x}, \tilde{z}, \tilde{y})$  is a solution of the LP relaxation at the current node of the branch-and-bound tree
- A digraph with root  $r$  is built using **node splitting**, every node  $i$  is split into two copies:
  - $i_{in}$  (entry)
  - $i_{out}$  (exit)
- The arcs and their capacities are defined as follows:

| Arc                          | Capacity      |
|------------------------------|---------------|
| $i_{in} \rightarrow i_{out}$ | $\tilde{z}_i$ |
| $r \rightarrow i_{in}$       | $\tilde{y}_i$ |
| $i_{out} \rightarrow j_{in}$ | $\infty$      |

*The arc with  $\infty$  capacity forces the flow to pass through the internal node arcs (with capacity  $\tilde{z}_i$ )*

# CORECON separation

## Fractional solution

- Consider each node  $\ell$  with  $\tilde{z} \geq \tau$
- The **min-cut** is found between the root and each node  $\ell$
- If the cut value is  $\leq \tilde{z}$ , then the CORECON inequality is violated
- The connectivity cut is defined as  $(\overline{W}_V, \overline{W}_A)$  such that:

$$\overline{W}_V = \{i \mid (i_{in}, i_{out}) \in \text{cut arcs}\}, \overline{W}_A = \{(r, i) \mid (r, i_{in}) \in \text{cut arcs}\}$$

- Lazy constraint added:

$$\sum_{i \in \overline{W}_V} z_i + \sum_{j \in \overline{W}_A} y_j \geq z_\ell, \quad \forall W \in W_\ell, \quad \forall \ell \in V$$

*In the implementation: the min-cut is computed using the **igraph** method `mincut`*

# CORECON separation

---

## Downlifting

- In both the fractional and integer case, **downlifting** is done
- Not all  $j \in W_A$  are considered, but only  $j \leq \ell$
- So all connected components are rooted to the node with smallest index

# COVER separation

- The **cover** will be the solution of a knapsack-problem:

$$\min \left\{ \sum_{j \in V_s} \tilde{z}_j q_j \mid \sum_{j \in V_s} w_j^s q_j \geq W_s - \lambda_s, \mathbf{q} \in \{0, 1\}^{|V_s|} \right\}$$

*To solve it, a heuristic is followed (greedy knapsack): sort nodes in non-decreasing way order of  $\tilde{z}_j/w_j^s$ ; greedily pick until  $\sum_{j \in C_s} w_j^s \geq W_s - \lambda_s$*

- Lazy constraints added:

$$\sum_{i \in C_s} z_i \geq u_s, \text{ if } s \in S_1$$
$$\sum_{i \in C_s} x_i \geq u_s, \text{ if } s \in S_2$$



# SCC separation

---

- Similar process to CORECON
- **Difference:** applied to the cover set  $C_s$  instead of a single node  $\ell$ 
  - An artificial **sink** node is added to the directed graph
  - For a certain specie  $s$ , every node in  $C_s$  is connected to the sink node
  - The min-cut is found between the root and the sink
- Lazy constraint added:

$$\sum_{i \in W_V} z_i + \sum_{j \in W_A} y_j \geq u_s, \quad \forall W \in W_s, s \in S_1$$

# Heuristics

---

# Construction heuristic

---

## Structure

- The construction heuristic generates a starting solution for initializing the branch-and-cut
  - **Phase 1:** creates a feasible solution in a greedy fashion
  - **Phase 2:** runs a post-processing to remove unnecessary nodes from  $S_z$
- The partial solution  $S$  is stored as  $(S_z, S_x)$  (core and reserve nodes) and  $W_s(S)$  (current habitat scores)

# Construction heuristic

---

- Let  $T(S)$  = set of **helpful** nodes not yet in  $S_z$ :  
a node is helpful if adding it to the core could protect at least one currently unprotected species

## Phase 1

1. Pick  $k$  random nodes as roots (one per component); add them to  $S_z$ , expand  $S_x$  with their buffer
2. **While** (PROTECT) not satisfied:
  - Find the shortest path from  $S_z$  to any node in  $T(S)$
  - Add all nodes on the path to the core; expand  $S_x$  with their buffers
  - Update  $T(S)$

*The computation of the shortest-path is done with the **NetworkX** Dijkstra algorithm `nx.multi_source_dijkstra`, weighted by a specific node-cost function that depends on suitability gain*

# Construction heuristic

---

## Phase 2

- Iterate through the nodes  $i \in S_z$  and check if, after removing  $i$ , the solution remains feasible
- The iteration is started with the node that causes the largest improvement in the objective function if removed
- Together with  $i$ , also the nodes from  $\delta_d(i)$  are removed, which become redundant after removing  $i$
- Repeat the process until no additional node can be removed

*The variables in the found solution are injected in the model as **Start** values*

# Primal heuristic

---

- Incorporated in the **branch-and-cut** framework (as a callback)

## Phase 1

- Essentially the same as the construction heuristic, but the **node-cost** function is changed to account for nodes already/partially selected in the LP solution

## Phase 2

- The same as the construction heuristic

*The found solution is injected via `cbSetSolution`*

# Local-branching heuristic

---

- Before the main MIP, the solution found by the construction heuristic is improved by adding the constraint:

$$\sum_{i \in S_z} z_i \geq |S_z| - r$$

- This searches for a better solution in the  $r$ -neighbourhood while ensuring that most of the core land parcels of the solution  $S$  also belongs to the new solution
- $r$  is incremented iteratively until a better solution is found or until the iteration limit is reached

*In the implementation the configuration chosen was:  $r = 5$ , time limit = 20s,  $\Delta_r = 5$ ,  $r_{max} = 20$*

# Results

---

## EXAMPLE

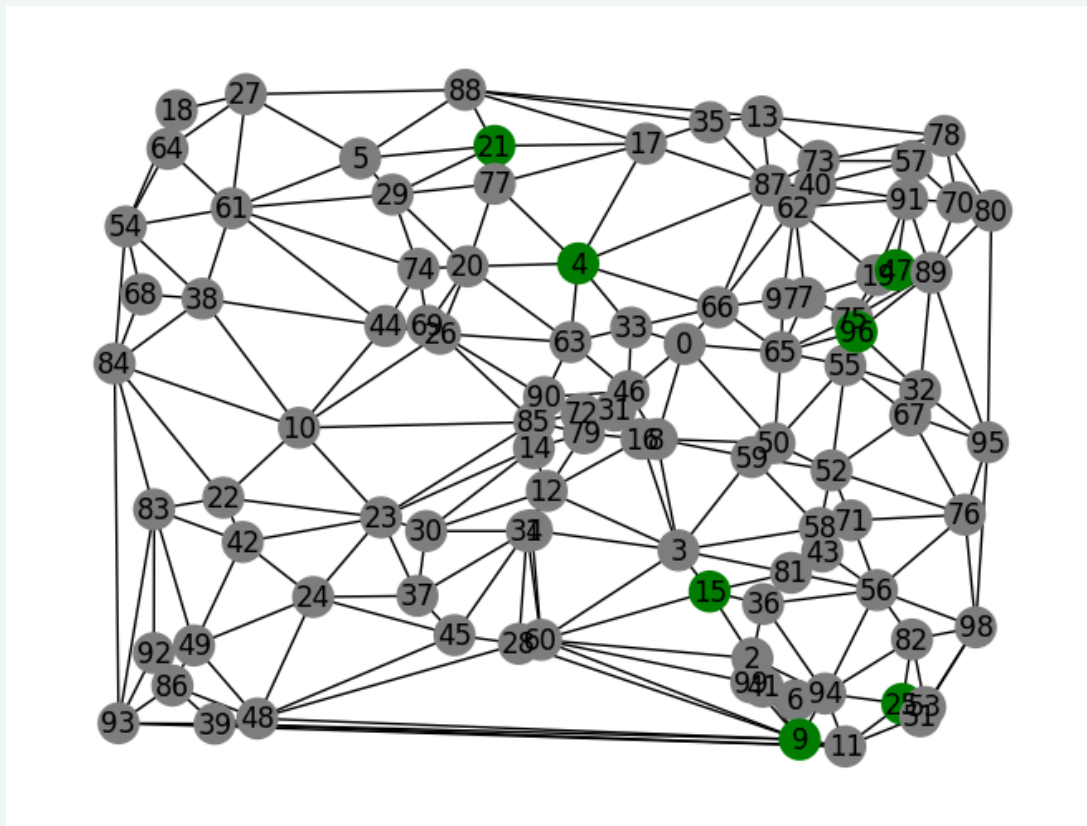
The system was tested on instances according to the quantities described in the paper

- Given
  - number of land parcels:  $n = 100$
  - number of species:  $m = 40$
  - number of max connected areas:  $k = 1$
  - buffer size:  $d = 1$
- The habitat suitability score is  $w_i^s \in [20, 100]$  and it's zero:
  - for the external nodes and  $s$  in  $S_1$
  - with probability 20% if the species  $s$  is in  $S_1$ , 10% if  $s$  in  $S_2$
- The cost of the land parcels is defined as  $c_i \in [1, 100]$
- The suitability quota is defined as:  $\lambda_s = \lceil 0.05 \rceil \sum_{i \in V_S} w_i^s$
- A random instance graph is generated using **Delaunay triangulation**

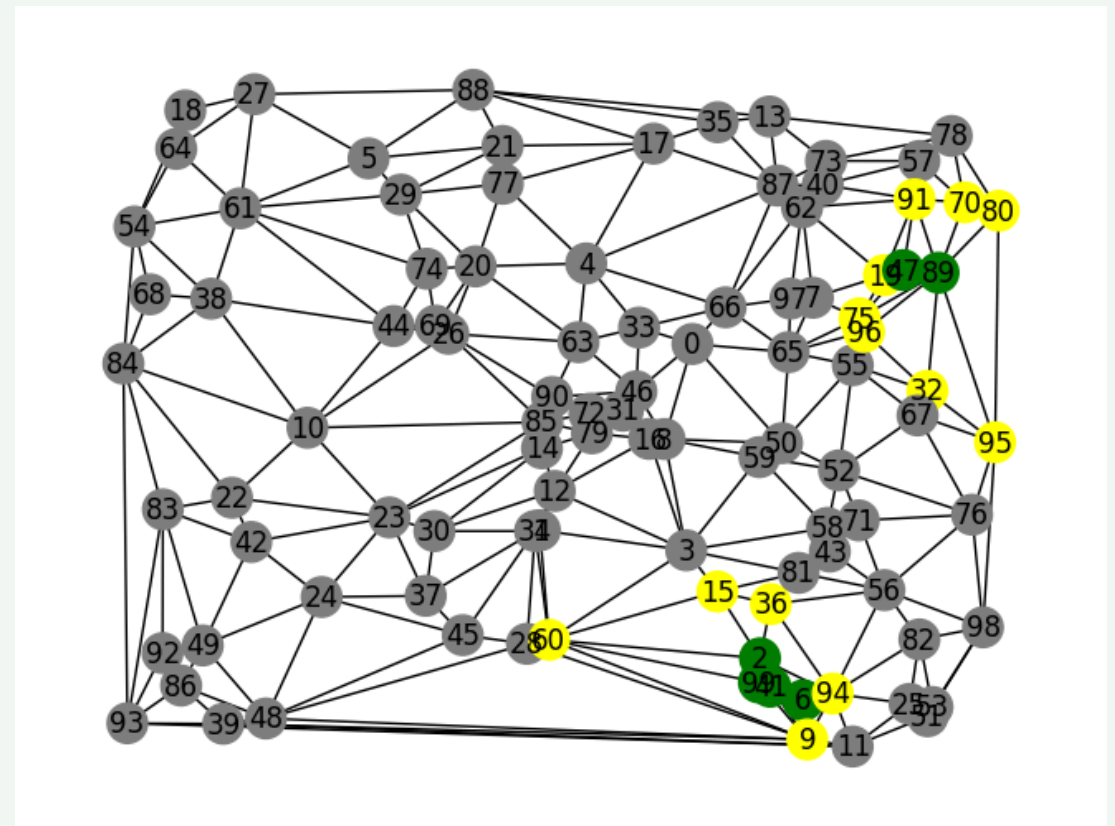


# Results

The GRSC model

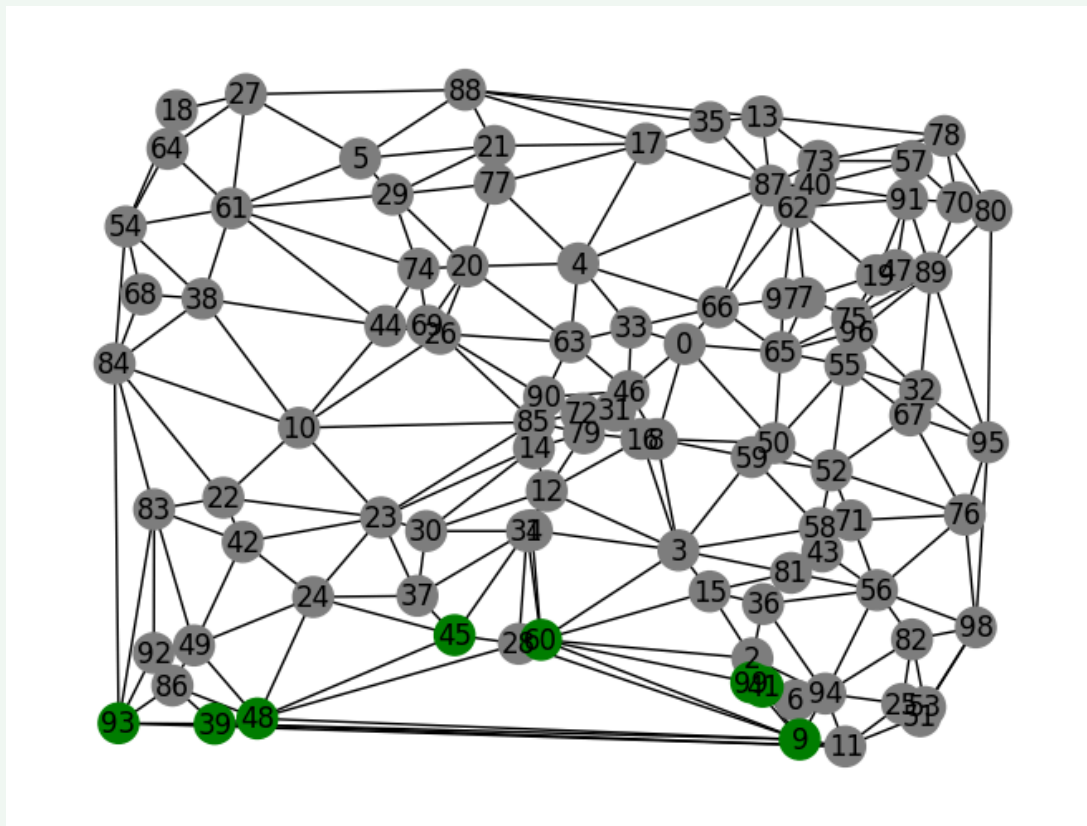


The GRSC-B model

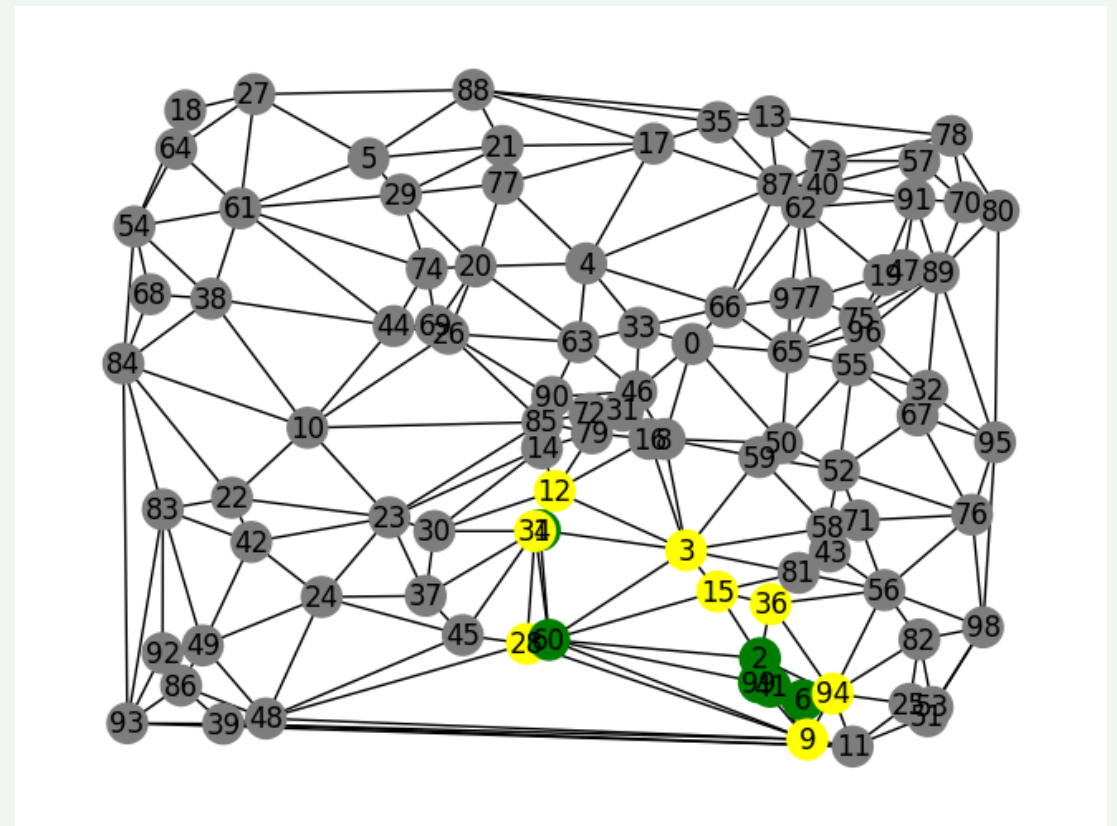


# Results

The GRSC-C model



The GRSC-CB model



# Results

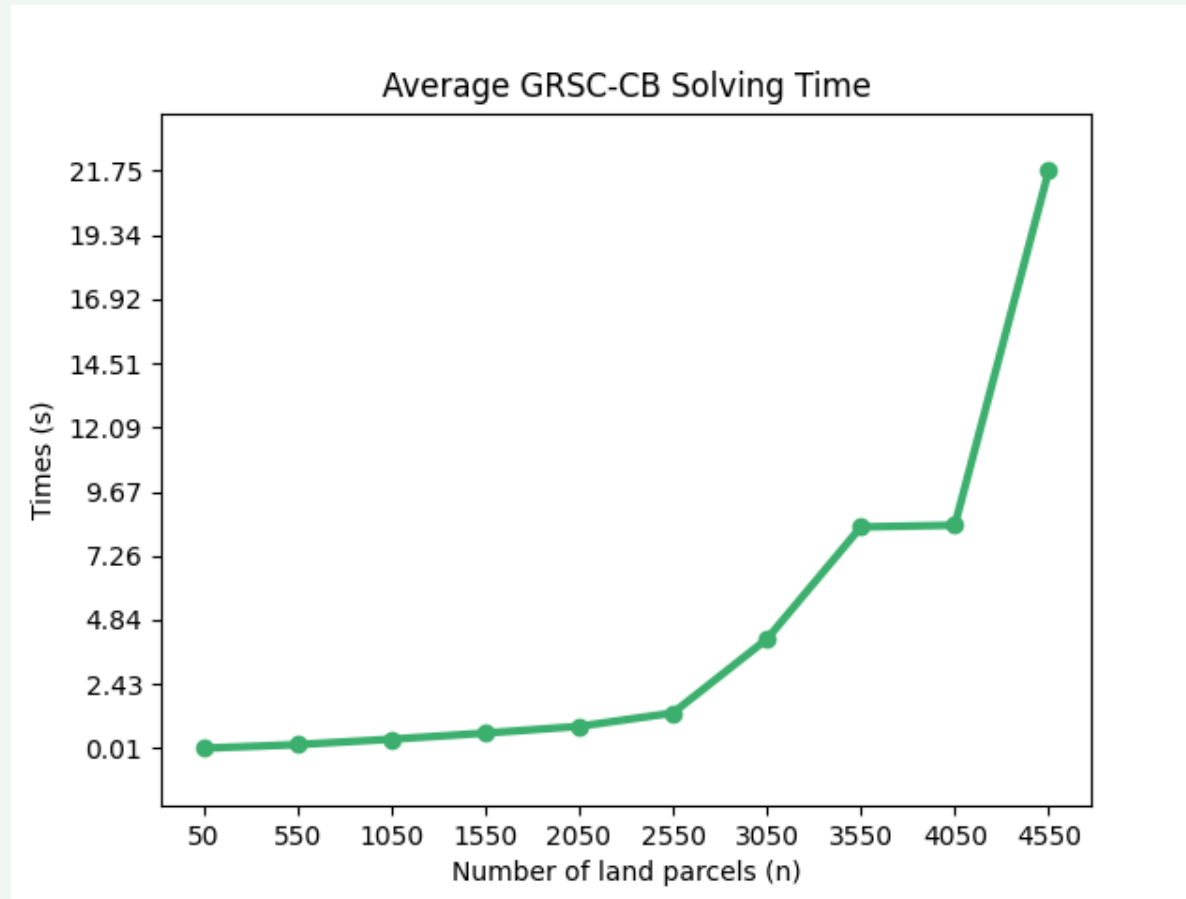
---

## SCALABILITY

- For the scalability test  $n$  was increased with:
  - $n_{min} = 100$
  - $n_{max} = 5100$
  - $n_{step} = 100$
- The solving time of the model with basic configuration was compared to the heuristics
- Results showed:
  - little to no improvement using the heuristics
  - an exponential increase of solving time starting from  $n = 3000$

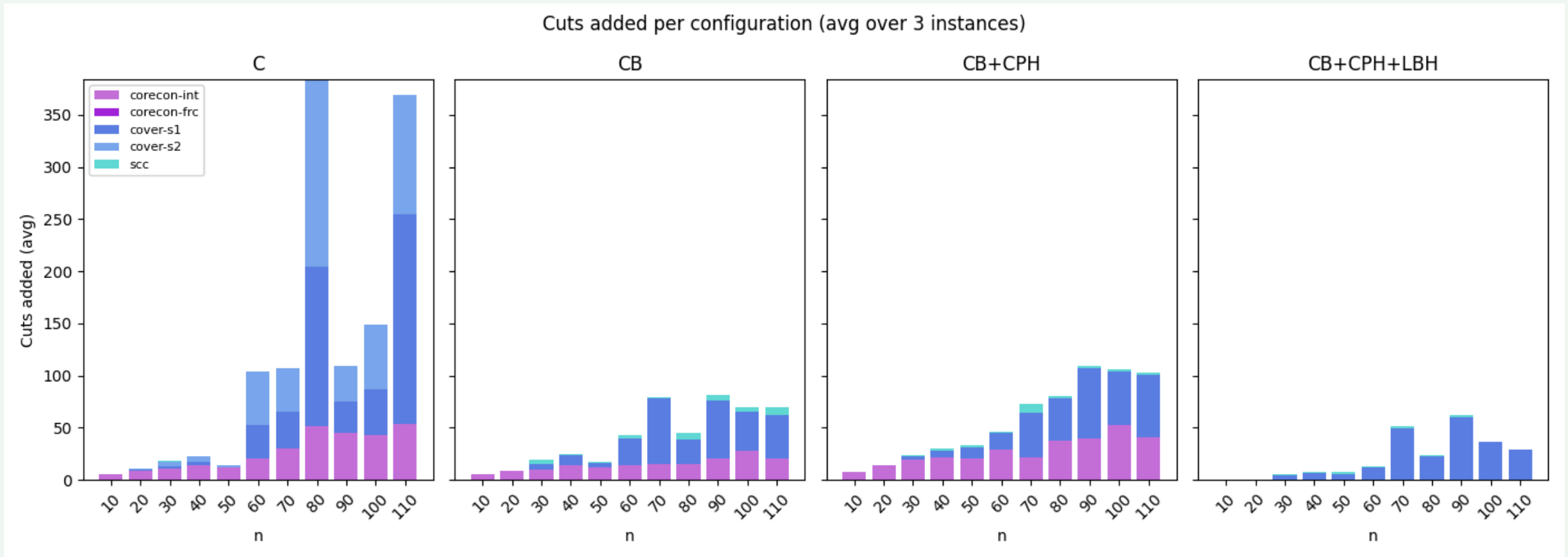
# Results

Average over 10 instances, for the complete configuration



# Results

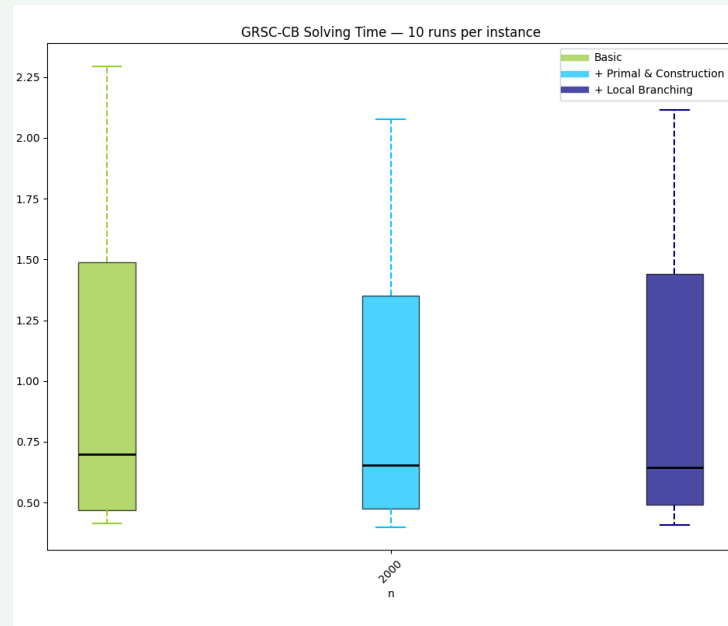
Amount of cuts added during the gurobi optimization, for 4 different configurations



# Results

## Heuristics

Average over 10 instances  
(n=2000)



## Solving times

Average over 3 instances, in the 3 possible configurations

